



PUCP

Taller de preparación para el Examen Parcial

*Layla Hirsh
Silvia Vargas
Diego Garate
Eric Huiza*



Objetivos del taller

- **Archivos:**
 - Reforzar el uso adecuado de las clases `ifstream` y `ofstream`
 - Reforzar el formateo de reportes, recordando qué manipuladores persisten y cuáles no.
 - Reforzar lectura formateada y no formateada.
 - Asegurar leer el archivo desde el inicio.
- **Arreglos:**
 - Reforzar la inserción ordenada
 - Reforzar el ordenamiento
 - Reforzar la búsqueda binaria

Archivos: `<fstream>` y la jerarquía de clases

- `<fstream>` agrega flujos para trabajar con archivos: `ifstream` (entrada) y `ofstream` (salida).



`ifstream` y `ofstream` heredan el comportamiento de `istream/ostream`: los operadores `>>` `<<` y el estado del flujo.

Las clases `ifstream` y `ofstream`

- El constructor o el método **`open(nombre)`** asocia el objeto con un archivo.
- **`is_open()`** indica si el archivo se abrió correctamente.
- **`eof()`** indica si la lectura llegó al final del archivo.
- **`close()`** cierra el archivo y libera el recurso.

Modos de apertura de un archivo y validación de apertura

- **ios::in** abre para lectura (modo por defecto de **ifstream**).
- **ios::out** abre para escritura (modo por defecto de **ofstream**); si el archivo existe, borra su contenido.

```
ifstream entrada("datos.txt");  
//o  
ifstream entrada;  
entrada.open("datos.txt");  
  
if (not entrada.is_open()) {  
    cout << "No se pudo abrir el archivo datos.txt" << endl;  
    exit(1);  
}
```

Errores comunes y buenas prácticas

- **No verificar `is_open()`** → el programa sigue ejecutándose como si el archivo existiera, produciendo resultados vacíos o incorrectos.
- **Confundir la ruta del archivo** → el programa busca el archivo en el directorio de ejecución, no en el del código fuente.

Formato de salida

- **fixed** fija la notación decimal, evitando notación científica.
- **setprecision(n)** define la cantidad de decimales mostrados.
- **setw(n)** reserva un ancho de columna fijo, útil para alinear reportes.
- **setfill(c)** define el carácter que rellena el espacio dejado por **setw**.
- **left** alinea el contenido a la izquierda de la columna; **right** (el valor por defecto) lo alinea a la derecha.
- **setfill(c)** cambia el carácter de relleno usado por **setw** — útil para separar columnas visualmente.
- **setw(n)** solo afecta a la siguiente salida; después de usarse, el ancho vuelve a su valor por defecto.
- **fixed**, **setprecision(n)**, **left/right** y **setfill(c)** permanecen activos hasta que se cambien explícitamente.

Lectura formateada vs. no formateada

- El operador `>>` salta espacios en blanco, saltos de línea y tabulaciones, interpreta el tipo del dato
- `get()` lee un caracter exactamente como está en el archivo, incluyendo espacios y saltos de línea, sin interpretar nada.

Leer un archivo varias veces, asegurar leerlo desde el inicio

```
archivo.clear();    // limpia el estado de fin de archivo (eof)
archivo.seekg(0);   // regresa el cursor al inicio
```

Tipos de archivo:

Tipo	Estructura	Ejemplos
TXT	Texto plano, sin reglas fijas de formato	2012336 JORGE-RAMIREZ
CSV	Texto plano, valores separados por comas	2012336, JORGE-RAMIREZ
JSON	Texto plano, pares clave-valor anidados	<pre>{ "codigo": 2012336, "nombre": "JORGE-RAMIREZ" }</pre>

Arreglos

- **Un arreglo** es una colección de datos del mismo tipo, guardados de forma contigua en memoria y accesibles con un único nombre.
- Cada dato ocupa una posición identificada por un índice, que empieza en 0 y llega hasta $n-1$ (n = tamaño del arreglo).
- Permite trabajar con muchos valores del mismo tipo usando un solo nombre y un ciclo.
- **Tamaño lógico** es cuántos datos hay guardados; **capacidad** es cuánto se reservó al declarar el **arreglo**. Ej.: en `notas[100]`, la capacidad es 100 aunque solo tengamos 30 datos leídos. Se necesita una variable que se actualice al agregar o quitar datos.

Insertar ordenado – versión 1:

```
bool insertarOrdenado(int* codigos, double* ciclos, int& tamaño,
                    int capacidad, int codigo, double ciclo) {
    if (tamaño >= capacidad) return false;

    int pos = buscarPosicionOrdenada(codigos, tamaño, codigo); // 1. buscar la posición
    for (int i = tamaño; i > pos; i--) { // 2. desplazar desde el final hasta la posición
        codigos[i] = codigos[i - 1];
        ciclos[i] = ciclos[i - 1];
    }

    codigos[pos] = codigo; // 3. colocar los valores
    ciclos[pos] = ciclo;
    tamaño++;
    return true;
}

int buscarPosicionOrdenada(int* codigos, int tamaño, int nuevo) {
    for (int i = 0; i < tamaño; i++)
        if (codigos[i] > nuevo)
            return i;
    return tamaño;
}
```

Insertar ordenado – versión 2

```
void insertarOrdenado(int* codigos, double* ciclos, int tamano, int codigo, double ciclo) {  
    int i;  
    for (i = tamano-1; i >= 0; i--) {// 1. buscar la posición y desplazar desde el final  
        if (codigo < codigos[i]){  
            codigos[i+1] = codigos[i];  
            ciclos[i+1] = ciclos[i];  
        }  
        else  
            break;  
    }  
    codigos[i+1] = codigo; // 2. colocar los valores  
    ciclos[i+1] = ciclo;  
}
```

Ordenación por intercambio – un criterio

```
void intercambiar(int &a, int &b) {
    int aux = a;
    a = b;
    b = aux;
}

// Función de ordenamiento por intercambio (ascendente)
void ordenamientoIntercambio(int *arr, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            // Si el elemento anterior es mayor que el actual, los intercambiamos
            if (arr[i] > arr[j]) {
                intercambiar(arr[i], arr[j]);
            }
        }
    }
}
```

Ordenación por intercambio – multicriterio

```
// Función de ordenamiento por intercambio multicriterio ascendente arr1 y arr2

void ordenamientoIntercambio(int *arr1, int *arr2, double *arr3, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            // Si el elemento anterior es mayor que el actual, los intercambiamos
            if (arr1[i] > arr1[j] or
                (arr1[i] == arr1[j] and arr2[i] > arr2[j])) {
                intercambiarInt(arr1[i], arr1[j]);
                intercambiarInt(arr2[i], arr2[j]);
                intercambiarDouble(arr3[i], arr3[j]);
            }
        }
    }
}
```


Búsqueda binaria: implementación

Requisito: arreglo ordenado ascendente (para este caso)

```
int buscarBinaria(int* arreglo, int cant, int buscado)
{
    int limiteInf = 0;
    int limiteSup = cant - 1;

    while (true) {
        int puntoMedio = (limiteInf + limiteSup) / 2;
        if (arreglo[puntoMedio] == buscado) {
            return puntoMedio;
        }
        if (arreglo[puntoMedio] < buscado) {
            limiteInf = puntoMedio + 1; // derecha
        } else {
            limiteSup = puntoMedio - 1; // izquierda
        }
        if (limiteInf > limiteSup) return NO_ENCONTRADO;
    }
}
```

Iter. 1



16 < 23, busca a la derecha

Iter. 2



42 > 23, busca a la izquierda

Iter. 3



23 == 23, encontrado

 medio comparado  descartado

3 comparaciones. La secuencial necesitaría 5.

El ciclo termina cuando `limiteInf` supera a `limiteSup`: ya no queda ningun candidato. Devuelve -1, igual que la secuencial.